POGS-ACT: Persistent Object Gaussian Splatting for Action Chunking Transformers

Student Name: Nischay Khade, Parth Goyal, Siddharth Yadav
Roll Number: 2023352, 2023371, 2023525

BTP report submitted in partial fulfilment of the requirements
for the Degree of B.Tech. in Computer Science & Engineering

on April 21, 2026

BTP Track: Research

BTP Advisor
Prof. Sanjit K. Kaul

Indraprastha Institute of Information Technology
New Delhi

Student Declaration

We hereby declare that the work presented in the report entitled **"POGS-ACT: Persistent Object Gaussian Splatting for Action Chunking Transformers"** submitted by us for the partial fulfilment of the requirements for the degree of *B.Tech. in Computer Science & Engineering* at Indraprastha Institute of Information Technology, Delhi, is an authentic record of my work carried out under the guidance of **Prof. Sanjit K. Kaul** . Due acknowledgements have been given in the report for all material used. This work has not been submitted elsewhere for the reward of any other degree.

Nischay Khade, Parth Goyal, Siddharth Yadav **Place & Date:** April 21, 2026

Certificate

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Prof. Sanjit K. Kaul **Place & Date:** April 21, 2026

Abstract

Autonomous robotic manipulation remains a formidable challenge in robotics, primarily due to the limitations of existing perception systems in handling visual occlusion, viewpoint sensitivity, and high data demands. This report presents **POGS-ACT**, a novel architecture that integrates **Persistent Object Gaussian Splatting (POGS)** with an **Action Chunking Transformer (ACT)** policy to address these limitations in the context of imitation learning.

Our approach replaces standard 2D camera feeds with an explicit, object-centric 3D Gaussian representation of the workspace. Semantic and visual features from foundation models (DINO, CLIP, Detic) are distilled into 3D Gaussians, from which structured point clouds are extracted and encoded via a PointNet++ encoder. This provides the policy with a clean, viewpoint-invariant geometric input. The ACT policy then predicts temporally smoothed sequences of joint-space actions, mitigating compounding execution errors.

POGS-ACT is architecturally designed to be significantly more data-efficient and computationally lighter than competing 3D manipulation approaches such as ManiGaussian and Gaussian World Models (GWM), while offering greater robustness to occlusion and viewpoint shift compared to 2D methods such as standard ACT. We also describe ongoing work toward language-conditioned task generalization, leveraging the CLIP-enriched point clouds already produced by POGS to enable multi-task scaling without retraining the perception pipeline.

The project is implemented on the **OpenManipulator-X** (4-DoF) robotic arm at IIIT-Delhi, using the LeRobot framework for data collection and training.

Keywords: Robotic Manipulation, Imitation Learning, 3D Gaussian Splatting, POGS, Action Chunking Transformers, PointNet++, Occlusion Robustness, Visuomotor Policy.

Acknowledgment

We would like to express our deepest gratitude to our project advisor, Prof. Sanjit K. Kaul, for his invaluable guidance, continuous encouragement, and insightful feedback throughout the course of this project. His mentorship was instrumental in navigating the complexities of data-driven robotics and refining our implementation of Action Chunking Transformers.
We also extend our sincere thanks to the Indraprastha Institute of Information Technology, Delhi (IIIT-Delhi) for providing us with the necessary resources and laboratory facilities, including the OpenManipulator-X robotic arm and computational support, which were essential for the successful completion of this work.

Contents

Chapter 1

Introduction

The ability of robots to autonomously perform fine-grained manipulation tasks in unstructured environments is a central goal of modern robotics. Imitation learning, wherein a robot learns by observing human demonstrations, has emerged as a promising and practical paradigm. However, deploying such systems in the real world continues to pose significant challenges rooted in how robots perceive their environment.

Most existing manipulation policies rely on 2D camera feeds as their primary perceptual input. While these approaches have achieved impressive results in controlled conditions, they are fundamentally fragile: performance degrades significantly when objects become occluded, when the camera viewpoint shifts, or when workspace lighting varies. More fundamentally, 2D images encode a large quantity of task-irrelevant background information, forcing the policy network to simultaneously learn visual feature extraction and motor control from raw pixels [4].

Three-dimensional perception methods offer a path forward, but current approaches introduce their own limitations. Voxel-based methods and dense scene reconstructors are computationally expensive [3]. Gaussian world models require massive demonstration datasets to capture global scene dynamics [5, 2]. Novel view synthesis approaches demand intensive multi-camera hardware setups and manual viewpoint selection for every task [3].

This report presents **POGS-ACT**, which proposes a principled decomposition of the perception and control problems. We use **Persistent Object Gaussian Splatting (POGS)** [8] as a persistent, object-centric geometric memory of the workspace, extract task-relevant point clouds from that representation, encode them with PointNet++ [6], and predict smooth action sequences with an **Action Chunking Transformer (ACT)** [10]. The key insight is that by decoupling perception from policy learning, and by focusing perception only on the task-relevant objects, we dramatically reduce the state-space complexity and data requirements while achieving robustness to occlusion and viewpoint change.

1.1 Problem Statement

The central challenge addressed in this project is the difficulty of generalization in autonomous robotic manipulation. We identify four key sub-problems that motivate this work:

- **Viewpoint Sensitivity:** Standard 2D imitation learning policies are tightly coupled to specific camera placements. A shift in camera position or angle causes significant performance degradation, as the learned visual features are not viewpoint-invariant.
- **Visual Occlusion:** Visual occlusion is a primary failure mode in dexterous manipulation. Targets become hidden either through external clutter or through the robot's own body (self-occlusion) [1, 11]. Adding more cameras is a poor solution, as it scales badly in complex multi-robot environments.
- **Perceptual Noise:** Most methods feed raw sensor streams directly to the policy. In any given task, the majority of the perceived workspace is task-irrelevant background. This forces the policy to devote model capacity to filtering noise rather than learning manipulation.
- **Temporal Inconsistency:** Predicting actions one step at a time often results in jerky, jittery motion. The problem lies in generating smooth, temporally consistent trajectories that resemble natural human motion rather than disjointed robotic steps.

1.2 Project Goal

The primary goal of this project is to implement and validate the POGS-ACT architecture on the OpenManipulator-X robotic platform, demonstrating that:

- An explicit, object-centric 3D representation can replace dense multi-camera setups while providing superior occlusion robustness and viewpoint invariance.
- Point clouds extracted from 3D Gaussians, encoded via PointNet++, provide a more informative and compact policy input than raw RGB images.
- An ACT policy conditioned on such representations can learn precise manipulation from fewer demonstrations than competing approaches.
- The architecture can be extended toward language-conditioned multi-task generalization without retraining the perception pipeline.

Chapter 2

Literature Review & Theoretical Background

This chapter reviews the body of work that directly informs and motivates the design of POGS-ACT. We trace the evolution of visuomotor policy learning for robotic manipulation, from standard imitation learning through 3D perception methods to 3D Gaussian Splatting-based approaches, and place each contribution in relation to our architecture.

2.1 Imitation Learning and Behaviour Cloning

Imitation learning, and specifically Behavioural Cloning (BC), is the dominant paradigm for teaching robots manipulation from human demonstrations. BC trains a policy $\pi_\theta(a_t \mid o_t)$ by supervised learning over a dataset of (o_t, a_t) pairs collected from a human expert. While conceptually simple, naive BC suffers from compounding errors: small deviations from the training distribution at each step accumulate, eventually causing the policy to enter states far from those seen during training. This distribution shift is the central challenge that Action Chunking Transformers directly address.

2.2 Action Chunking Transformers (ACT)

The foundational method for this project is ACT [10]. ACT reformulates the policy as a Conditional Variational Autoencoder (CVAE) that predicts a *chunk* of k future actions rather than a single action. This has two key benefits: the policy captures the temporal structure and smoothness of human demonstrations, and the effective control frequency is reduced, reducing compounding errors.

2.2.1 Architecture

The ACT encoder processes visual observations (RGB images from 2–4 cameras) via CNNs and proprioceptive joint angles via MLPs. A Transformer encoder models spatial-temporal dependencies across these modalities. A Transformer decoder outputs a predicted action sequence of length k.

2.2.2 Temporal Ensembling

At inference, the policy is queried at every timestep, generating overlapping chunks. The executed action is a weighted average of all predictions covering the current time:

$$a_t = \sum_i w_i \cdot \hat{a}_t^{(i)} \tag{2.1}$$

where $\hat{a}_t^{(i)}$ is the prediction for time t from query at step i, and w_i is an exponentially decaying weight. This temporal ensembling acts as a low-pass filter, producing smooth, stable trajectories.

2.2.3 Latent Style Variable

To handle multimodal demonstrations (multiple valid ways to complete a task), ACT incorporates a latent style variable z sampled from the CVAE encoder during training and set to zero during inference. This prevents the decoder from averaging over conflicting modes.

Limitations of standard ACT: The policy is tightly coupled to fixed camera placements. It is sensitive to viewpoint changes, cannot reason about occluded geometry, and requires ∼50 demonstrations per task as the visual encoder must learn representation and control simultaneously.

2.3 Robotic View Transformer (RVT)

RVT [3] addresses viewpoint sensitivity by synthesizing novel virtual perspectives from existing RGB-D camera feeds. Sensor data is re-rendered into strategically chosen virtual viewpoints around the workspace, and a Transformer predicts the next keyframe end-effector pose from these synthetic views. A low-level motion planner executes the predicted trajectory. RVT effectively "sees around" physical occlusions without building an explicit 3D representation.

Limitations: RVT requires 4 RGB-D cameras, significant compute for rendering, and manual selection of optimal virtual viewpoints for each new task.

2.4 3D Diffusion Policy (DP3)

DP3 [9] introduces explicit 3D structure into the diffusion policy framework. A single-view depth image is encoded as a sparse point cloud via a lightweight MLP encoder. A diffusion model denoises

Gaussian noise into action sequences conditioned on the 3D point cloud features and robot joint states. DP3 demonstrates that 3D inputs provide a strong inductive bias for manipulation.

Limitations: Single-view depth limits occlusion handling. The MLP point cloud encoder lacks PointNet++'s hierarchical local geometry capture.

2.5 ManiGaussian

ManiGaussian [5] lifts manipulation into a full 3D Gaussian Splatting representation. Expert demonstrations build a dynamic 3D Gaussian scene enriched with Stable Diffusion semantic features. A Gaussian world model is trained with a self-supervised future-scene-prediction objective, and a PerceiverIO Transformer predicts the next keyframe end-effector pose.

Limitations: The representation is scene-centric: the network models the entire global environment at every timestep, including task-irrelevant background. This creates data bottlenecks (100+ demonstrations required) and high inference latency. Gaussians are compressed into a latent space, discarding explicit geometric structure.

2.6 Gaussian World Models (GWM)

GWM [2] extends ManiGaussian toward scalability by compressing the Gaussian scene representation into a compact latent vector for real-time training and inference. While faster, GWM retains the scene-centric framing and inherits the same data bottleneck: the latent must encode global scene dynamics, not just the task-relevant geometry.

2.7 Point Clouds as Policy Inputs

A rigorous empirical study [4] benchmarks the effect of different observation modalities on robot learning across a range of tasks. The central finding is that explicit point clouds consistently outperform raw RGB images as policy inputs. The authors attribute this to the preservation of Euclidean 3D geometry and 3D neighbourhood locality, properties that 2D projections inherently discard. This empirical evidence directly motivates the POGS-ACT design choice of routing Gaussian centers through PointNet++ rather than compressing them into an opaque latent.

2.8 PointNet++

PointNet++ [6] is a hierarchical deep learning architecture for point sets. It groups points at multiple spatial scales, applies a shared MLP encoder at each level, and progressively aggregates local features into a global geometric descriptor. Unlike global pooling approaches, PointNet++ preserves dense, local structural details of object geometry, which are critical for contact-rich tasks where the robot must reason about edges, gaps, and surface normals.

2.9 Persistent Object Gaussian Splatting (POGS)

POGS [8] creates an object-centric, persistent digital twin of the workspace using 3D Gaussian Splatting. Three foundation models are combined: **Detic** for open-vocabulary instance segmentation, **DINO** for robust self-supervised visual features, and **CLIP** for natural language-aligned semantic embeddings. These features are distilled into per-object 3D Gaussians that persist across time and update continuously as new observations arrive. POGS tracks object geometry under severe dynamic occlusion without pausing to rescan, and requires no prior CAD models.

The critical property for POGS-ACT is that POGS provides a persistent, explicit 3D geometric memory of each object, indexed by semantic identity, from which clean point clouds can be extracted at any time.

2.10 LangSplat

LangSplat [7] demonstrates 3D language fields by training per-scene language autoencoders on CLIP features and baking them into 3D Gaussians. This enables open-vocabulary querying of 3D scenes via natural language. We review LangSplat as a reference for our ongoing work on language-conditioned task generalization, where we intend to use the CLIP embeddings already present in the POGS map to query task-relevant object point clouds via text prompts.

2.11 Summary of Related Work

Table 2.1 summarises the key dimensions along which POGS-ACT is positioned relative to existing methods.

Table 2.1: Comparison of manipulation policy approaches.

Method	**3D Repr.**	**Obj.-centric**	**Occl. robust**	**Cameras**	**Demos**
ACT [10]	–	–	Low	2–4	∼50
RVT [3]	Virtual	–	Medium	4	∼100
DP3 [9]	Point cloud	–	Low	1	∼50
ManiGaussian [5]	Gaussian	–	High	4	100+
GWM [2]	Gaussian	–	High	4	100+
POGS-ACT (ours)	**Gaussian**	**Yes**	**High**	**1**	**<50**

Chapter 3

Methodology & System Design

This chapter details the POGS-ACT architecture and experimental setup. We describe each component of the pipeline, explain the design decisions that distinguish our approach from existing methods, and document the hardware and software environment used for implementation.

3.1 Architectural Overview

POGS-ACT consists of two tightly coupled stages. The **perception stage** uses POGS and PointNet++ to transform raw RGB-D observations into a compact, viewpoint-invariant geometric latent. The **control stage** uses ACT to map this latent and the robot's joint state to a smooth sequence of target joint angles.

The key architectural principle is **decoupling**: representation learning is handled entirely by the pre-trained foundation models within POGS, and the ACT policy only sees a clean geometric descriptor of the task-relevant object. This removes the burden of visual feature learning from the policy, reducing the required demonstration count and simplifying the learning problem.

Figure 3.1 shows the early-stage rough design sketch of the pipeline, illustrating the data flow from the scanning camera through Gaussian map construction, object querying, and PointNet++ encoding into the ACT policy. Figure 3.2 shows the formal system architecture as presented in the project overview, with the POGS and ACT sub-systems clearly delineated.

3.2 Stage 1: Perception via POGS

3.2.1 Scene Scanning

Prior to task execution, the robot arm follows a predefined scanning trajectory through the workspace while a single Intel RealSense D455 camera captures RGB-D frames from multiple viewpoints. This is a one-time setup step; the resulting Gaussian map persists and is only incrementally updated during task execution.

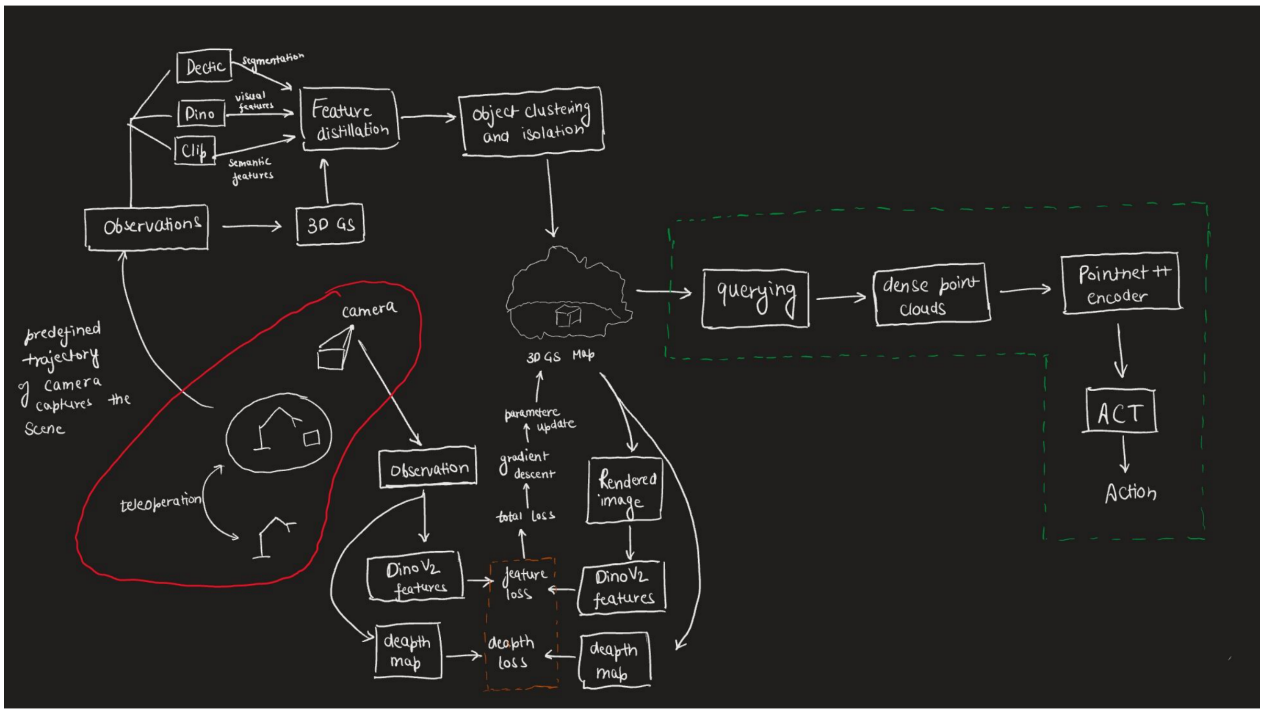

Figure 3.1: Rough architecture sketch of the POGS-ACT pipeline. Left: the teleoperation and scanning loop that builds the 3D Gaussian map from predefined camera trajectories. Centre: the POGS pipeline, showing the Gaussian optimisation loop (DINOv2 feature loss, depth loss) and the object clustering step. Right (dashed green box): the ACT control pipeline, from object querying through dense point cloud extraction, PointNet++ encoding, and action generation.

3.2.2 Foundation Model Feature Extraction

Three pre-trained foundation models are applied to each captured frame:

- **Detic:** Open-vocabulary instance segmentation. Identifies and delineates each object in the workspace from the background, without requiring task-specific retraining.
- **DINO:** Self-supervised vision transformer. Extracts dense, robust visual features that are viewpoint-consistent and semantically structured.
- **CLIP:** Vision-language model. Extracts natural language-aligned semantic embeddings, enabling text-based querying of the resulting 3D representation.

3.2.3 Feature Distillation into 3D Gaussians

The 2D features from all three models are distilled into the 3D Gaussian Splatting representation via a feature distillation step. This creates a rich semantic "Feature Field" embedded in 3D space, where each Gaussian primitive carries both geometric parameters (position, covariance, opacity) and semantic feature vectors. The Gaussian map is optimised using a composite loss:

$$\mathcal{L} = \lambda_{\text{rgb}}\mathcal{L}_{\text{rgb}} + \lambda_{\text{dino}}\mathcal{L}_{\text{dino}} + \lambda_{\text{depth}}\mathcal{L}_{\text{depth}} \tag{3.1}$$

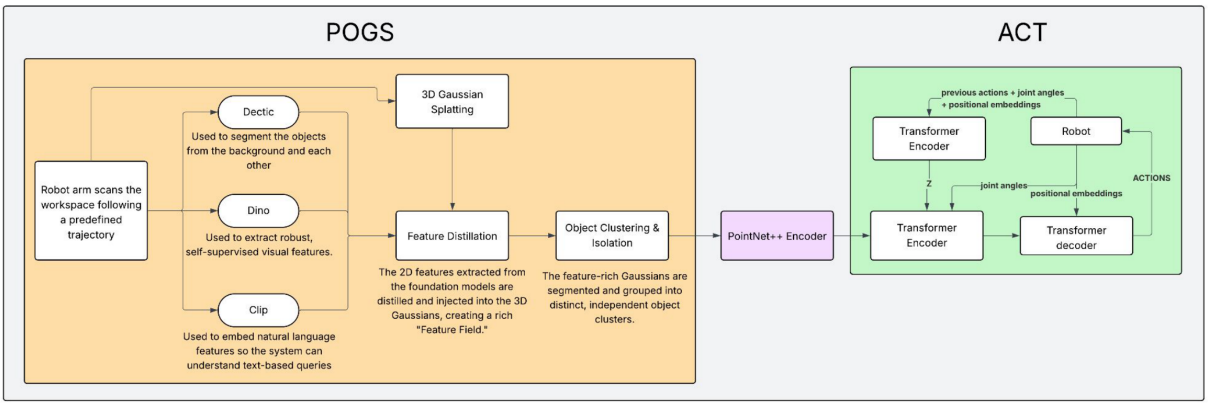

Figure 3.2: Formal POGS-ACT system architecture. The POGS stage (orange, left) processes observations from the scanning robot arm through Detic, DINO, and CLIP to build a 3D Gaussian Splatting map, followed by feature distillation and object clustering. The resulting object cluster is passed to the PointNet++ encoder (purple), whose spatial latent z conditions the ACT Transformer encoder–decoder (green, right) to produce joint-space actions.

where $\mathcal{L}_{\text{rgb}}$ is the standard photometric reconstruction loss, $\mathcal{L}_{\text{dino}}$ enforces feature consistency between rendered and extracted DINO features, and $\mathcal{L}_{\text{depth}}$ enforces geometric accuracy.

3.2.4 Object Clustering and Isolation

Once the Gaussian map is constructed, POGS segments it into distinct per-object clusters using the Detic segmentation masks as guidance. The CLIP embeddings within each Gaussian allow the system to identify objects by natural language query (e.g., "red cube"). For a given task, the relevant object cluster is selected and its Gaussian centers are extracted as an explicit 3D point cloud. This step is the mechanism by which POGS-ACT achieves its object-centric focus: the downstream policy never sees the full scene.

3.3 Stage 2: Point Cloud Encoding via PointNet++

The extracted Gaussian centers form a 3D point cloud $\mathcal{P} \in R^{N\times 3}$. This is passed through a PointNet++ encoder [6]. The hierarchical architecture:

1. Groups nearby points at multiple spatial scales using farthest point sampling and ball-query grouping.

2. Applies shared MLP layers to extract local geometric features at each scale.

3. Progressively aggregates local features into a global geometric descriptor.

The output is a compact spatial latent vector $z \in R^d$ that encodes the 3D geometry of the target object. By routing raw Gaussian centers (pure Euclidean coordinates) through PointNet++, we preserve 3D neighbourhood locality and capture the dense local structural details critical for precision manipulation [4].

3.4 Stage 3: Action Generation via ACT

The ACT policy [10] takes the spatial latent z, the current robot joint angles q_t, and learned positional embeddings as input. A CVAE encoder maps the joint state to a style variable during training. A Transformer decoder, conditioned on z, the style variable, and joint embeddings, predicts a chunk of k future target joint angles:

$$\hat{a}_{t:t+k} = \pi_\theta(z, q_t) \tag{3.2}$$

At inference, consecutive chunks are averaged with exponential weighting (Temporal Ensembling), producing smooth, physically consistent trajectories. The policy outputs 4-DoF target joint angles directly, bypassing Inverse Kinematics entirely and guaranteeing that all predicted configurations are physically reachable by the OpenManipulator-X.

3.5 Hardware Setup

3.5.1 Robotic Manipulator

We use the **OpenManipulator-X (RM-X52-TNM)**, a 4-DoF robotic arm compatible with ROS interfaces. This platform was chosen for its accessibility, open-source ecosystem, and suitability for validating data-efficient manipulation policies on low-cost hardware.

3.5.2 Vision System

Two camera views are used:

- **Wrist View:** An Intel RealSense D455 mounted on the end-effector. Provides a dynamic, close-up view of the manipulation target and supplies the RGB-D stream for POGS scanning and object tracking.
- **Static Front View:** A stationary camera positioned to capture the global workspace context.

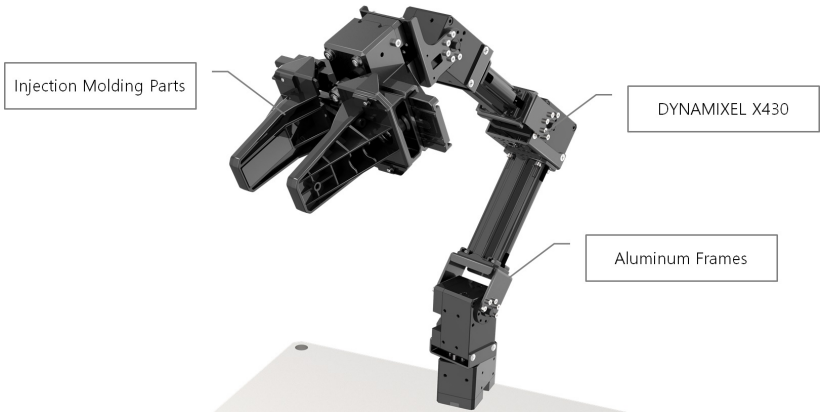

Figure 3.3: The OpenManipulator-X (RM-X52-TNM) used for all experiments.

Figure 3.4: Intel RealSense D455 camera used for both POGS scanning and live observation.

3.6 Environment Engineering

To ensure robust policy learning and reduce visual noise, we implemented the following physical workspace optimisations:

- **Workspace Isolation:** Physical enclosure panels eliminate background distractions from the camera field of view.
- **Visual Contrast Enhancement:** Red tape applied to the gripper jaws provides a strong colour contrast for the wrist-view camera.
- **Background Consistency:** A dark, uniform surface maximises contrast between the target object and the table.
- **Uniform Lighting:** Diffuse lighting eliminates harsh shadows that caused false positive detections in initial experiments.

3.7 Software Framework

Data collection, training, and deployment are implemented within the **LeRobot** framework, which provides abstractions for teleoperation, dataset management, and model training. The POGS pipeline runs as a separate ROS node that maintains and updates the Gaussian map. The PointNet++ encoder and ACT policy are implemented in PyTorch.

Chapter 4

Implementation & Data Collection

4.1 Data Collection Strategy

4.1.1 Dataset Composition

Initially, we have been testing this setup in simulation where we use the RLBench dataset and utilize the expert demonstrations given in the PerACT dataset as our primary training data source. First, for every timestep that we have RGB images, we conduct a screen capture of the RLBench scene. Here, we do this for the Stack Blocks task. Now, using this Gaussian Map, we query the map at every timestep to move the Gaussians dynamically and track the clustered blocks as the robot performs the demonstration. During this, at every timestep, we export the Gaussian Map into a point cloud, embed the DINO features in each point, and then compress this point cloud using the PointNet++ encoder to generate an embedding for ACT. So, if the episode was for 100 timesteps, then there are 100 embeddings comprising one episode of a task as training data for ACT.

4.1.2 Task Definition

The primary experimental task was **stack blocks**. To prevent the policy from memorising a fixed trajectory, the target object's starting position is randomised for every episode. This spatial randomisation is crucial for learning a policy that relies on visual and geometric cues rather than coordinate memorisation. The task is chosen to expose the limitations of 2D perception (occlusion, viewpoint shift) that POGS-ACT is designed to overcome, as illustrated by the dynamic simulator setup shown in Figure 5.2.

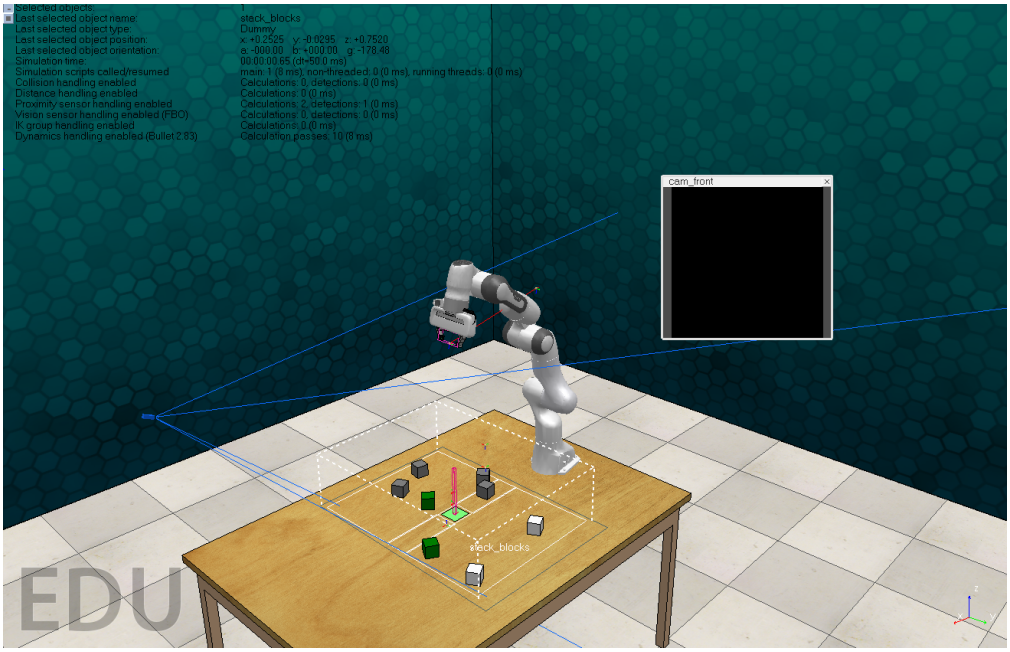

Figure 4.1: **Stack Blocks** task setup in CoppeliaSim with Franka Robot

4.2 POGS Scanning Procedure

In our simulated setup, the scene capture and processing procedure is executed continuously across the expert demonstrations rather than as a single static pre-scan:

1. **Scene Capture:** At every timestep, RGB images are captured directly from the RLBench simulation environment as the robot performs the task.
2. **Dynamic Tracking:** The Gaussian Map is queried at each timestep to dynamically move the Gaussians, effectively tracking the clustered blocks throughout the demonstration.
3. **Point Cloud & Feature Embedding:** At every timestep, the updated Gaussian Map is exported into a point cloud, and DINO features are embedded into each individual point.
4. **Compression & Encoding:** The feature-enriched point cloud is then compressed using a PointNet++ encoder to generate a single state embedding for the Action Chunking with Transformers (ACT) policy.

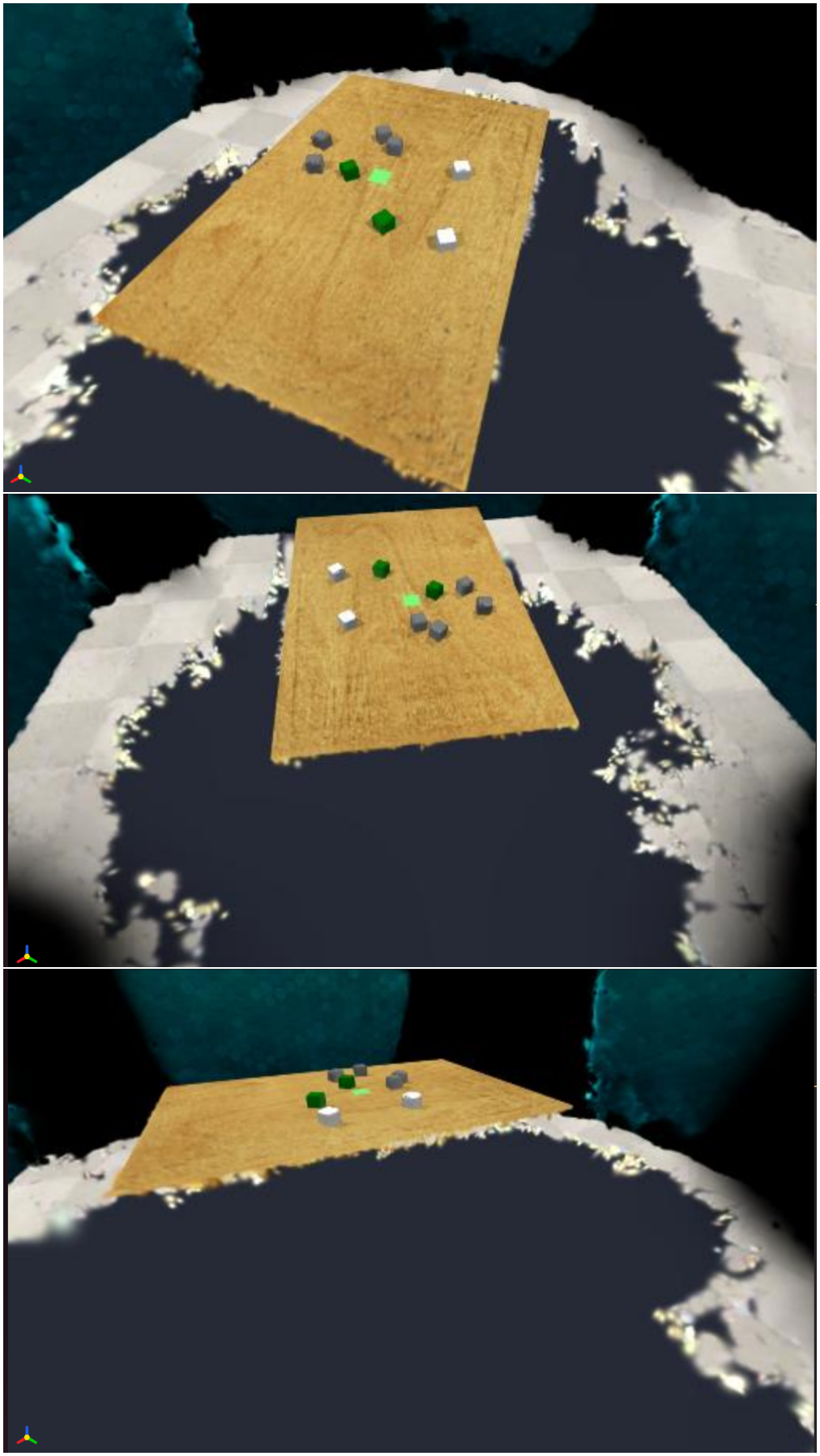

Figure 4.2: Different views of **Stack Blocks Task** Gaussian Map which is updated by the POGS-ACT framework as robot performs demonstration

Through this continuous procedure, an episode lasting T timesteps directly yields a sequence of T embeddings, which comprises the complete training data for that specific task demonstration.

4.3 Training Process

4.3.1 Input Pipeline

At each timestep during training, the system constructs the policy input as follows:

- The current POGS object cluster is queried to extract Gaussian centers, forming a 3D point cloud $\mathcal{P} \in R^{N\times 3}$.
- The point cloud is passed through the PointNet++ encoder to produce the spatial latent z.
- The current robot joint angles q_t are embedded via an MLP.
- z and the joint embedding are concatenated and passed to the ACT Transformer.

4.3.2 Policy Learning

The ACT model is trained via supervised learning to minimise the reconstruction loss between the predicted action chunk and the ground-truth expert trajectory:

$$\mathcal{L}_{\text{ACT}} = E\left[\sum_{j=0}^{k-1} \|a_{t+j} - \hat{a}_{t+j}\|^2\right] + D_{\text{KL}}\left(q(z \mid a_{t:t+k}, o_t) \,\|\, p(z)\right) \tag{4.1}$$

where the first term is the action reconstruction loss over the chunk and the second is the CVAE KL regularisation. The PointNet++ encoder is fine-tuned jointly with the ACT policy during training.

4.3.3 Training Configuration

- **Chunk size k:** 100 steps
- **Optimiser:** AdamW with cosine learning rate schedule
- **Batch size:** 8 episodes
- **Training epochs:** 1000
- **Point cloud size N:** 1024 points per object cluster

4.4 Inference and Deployment

At inference time, the POGS map is queried at each control cycle to extract the current object point cloud. The PointNet++ encoder produces the latent z, and the ACT decoder outputs a chunk of

100 target joint angles. The robot executes the first action of the chunk while the next chunk is computed in parallel. Temporal assembly is applied with an exponential weighting parameter $w = 0.1$ to smooth the final trajectory executed.

Chapter 5

Experiments & Results

This chapter presents the experimental validation of the POGS-ACT pipeline on the OpenManipulator-X. We describe the baselines against which we compare, the evaluation metrics, our initial experimental findings, and the environmental optimisations that led to a robust working policy.

5.1 Baselines

POGS-ACT is evaluated against the following six baselines, spanning the range of current state-of-the-art approaches:

1. **POGS** [8]: The perception module alone, used as a reference for geometric representation quality and occlusion tracking.
2. **Pi0.5**: A large-scale generalist robot policy, representing the upper bound of data-rich approaches.
3. **ACT (2-camera 2D)** [10]: Standard ACT with two fixed RGB cameras, the direct imitation learning baseline.
4. **ManiGaussian** [5]: Full Gaussian world model with scene-level reconstruction.
5. **RVT** [3]: Novel view synthesis-based 3D manipulation policy.
6. **DP3** [9]: 3D diffusion policy with sparse point cloud input.

5.2 Evaluation Metrics

Three primary metrics are used:

- **Task Success Rate:** Evaluated over high-precision, contact-rich pick-and-place trials with randomised object positions, focusing on tight-tolerance grasping. This is the primary measure of policy quality.

- **Compounding Execution Drift:** Measured as the deviation between the policy-generated trajectory and the ground-truth expert trajectory over extended temporal horizons, quantifying the degree to which temporal ensembling mitigates covariate shift.
- **Inference Latency:** Measured in milliseconds per closed-loop control cycle, comparing the computational cost of POGS-ACT against dense reconstruction approaches such as ManiGaussian and GWM.

5.3 Initial Experiments

Initial experiments were conducted with raw 2D camera inputs(with a depth sensor), to integrate the POGS perception pipeline. During this phase, we tested the robustness of the gaussian tracking and how across robot acts does the tracked gaussians move accurately enough for ACT to correctly manipulate the blocks and perform the task [10].

However, the system exhibited significant instability due to visual ambiguity. In numerous instances, the robot misinterpreted harsh shadows cast by the gripper which lead to mismatch between ground truth camera feed and rendered RGB image from the Gaussian Map with camera's viewpoint, leading to incorrect tracking done by POGS of the block's gaussians and eventually leading to poor grasping. These failures highlighted the model's sensitivity to lighting conditions and the difficulty of learning a visually grounded policy from raw 2D images alone.

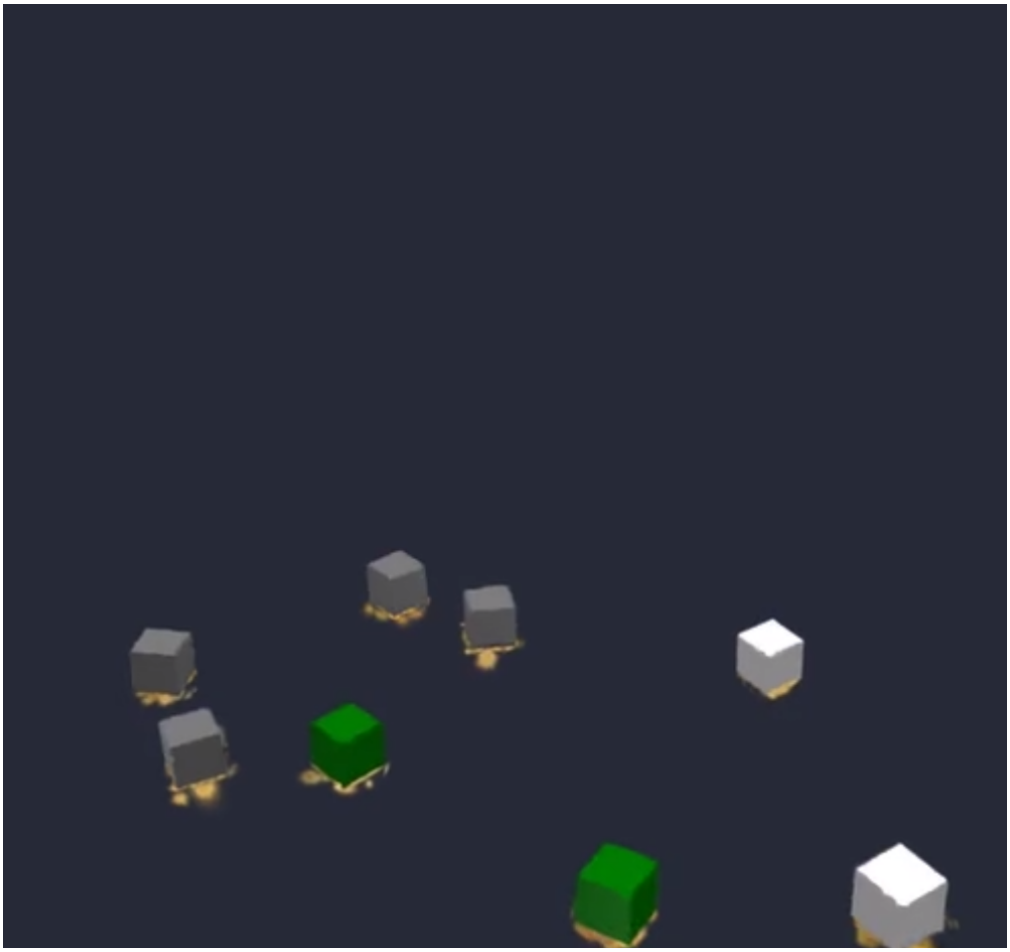

Figure 5.1: **Stack Blocks** task tracked gaussians clustered using HDBSCAN algorithm.These are the ONLY gaussians which are tracked and updated in the scene leading to the computational efficiency.

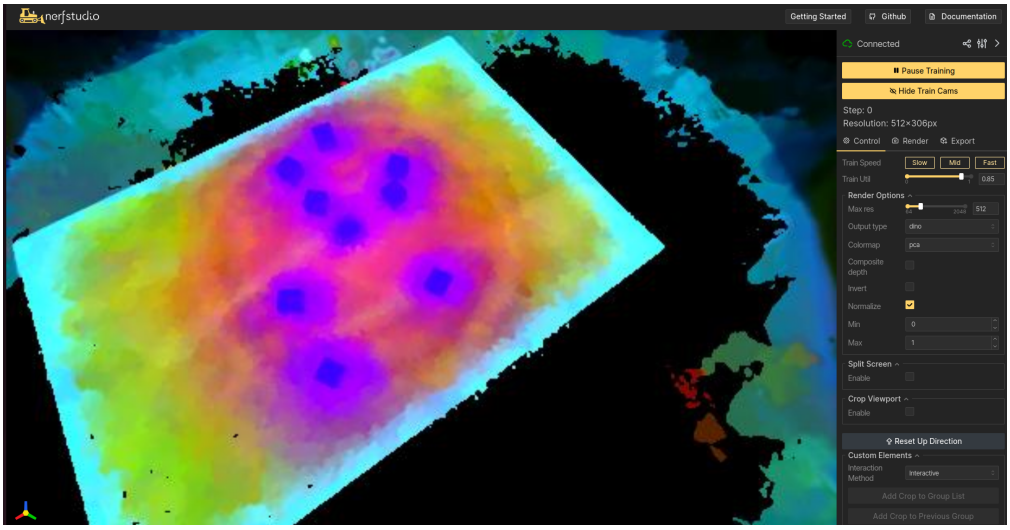

Figure 5.2: **Stack Blocks** Dino features projected from 2D image to 3D gaussian map to track the clustered blocks.

Chapter 6

Future Scope

The successful implementation of POGS-ACT on the OpenManipulator-X establishes a solid foundation for several advanced research directions. We outline five concrete avenues for extending this work.

6.1 Language-Conditioned Task Generalization

The current architecture trains a separate ACT module per task, limiting scalability. A key near-term goal is to exploit the CLIP embeddings already embedded in the POGS Gaussian map to enable language-conditioned task generalization.

The proposed mechanism is as follows: a natural language task description (e.g., "pick up the red mug") is encoded by CLIP's text encoder and used to query the Gaussian map by cosine similarity. Gaussians whose embeddings align with the text query are selected, producing a task-relevant point cloud without rescanning. Different ACT modules can then be activated by the text prompt, allowing the shared POGS map and PointNet++ encoder to serve multiple tasks. We are reviewing LangSplat [7] to establish the most principled approach for integrating language querying into the 3D Gaussian framework.

6.2 Virtual View Augmentation During Training

The POGS Gaussian map can render novel viewpoints of the workspace synthetically. During training, augmenting the fixed camera view with such rendered virtual views would expose the PointNet++ encoder to a wider distribution of object poses and viewpoints, potentially improving generalisation to novel configurations at test time. We also propose investigating an action-conditioned virtual camera policy: a lightweight network that predicts the optimal virtual viewpoint from which to observe the scene given the current task context.

6.3 Learnable Prompt Integration

Rather than using a fixed text query to select the relevant Gaussian cluster, we plan to investigate **learnable prompts**: task-specific embedding vectors optimised jointly with the ACT policy via gradient descent. This would allow the query mechanism to be fine-tuned for the specific geometric features relevant to each manipulation task, beyond what a hand-crafted text description can express.

6.4 Depth-Based Wrist Camera Synthesis

A practical limitation of the current setup is the wrist-mounted Intel RealSense D455, which adds weight and cable management complexity to the end-effector. In future iterations, we plan to eliminate the physical wrist camera by synthesising a virtual wrist-view depth image from the POGS Gaussian map. Given the robot's known forward kinematics at each timestep, the wrist camera pose is known, and the Gaussian map can render a depth image from that pose without hardware on the end-effector. This would reduce hardware dependencies and enable deployment on platforms where mounting a wrist camera is not practical.

6.5 Multi-Robot Scaling

The long-term goal of this research direction is to scale POGS-ACT to collaborative multi-robot systems. A single shared POGS map can track multiple objects and the geometry of multiple robots simultaneously. Separate ACT modules can coordinate different robots' actions conditioned on the same shared geometric representation. The key challenge is maintaining low-latency map updates when multiple robots are simultaneously modifying the scene, which we plan to address with asynchronous object-cluster update scheduling.

Chapter 7

Conclusion

This project presented **POGS-ACT**, a novel robotic manipulation architecture that integrates Persistent Object Gaussian Splatting [8] with an Action Chunking Transformer policy [10] to address the core limitations of existing imitation learning systems: viewpoint sensitivity, visual occlusion, perceptual noise, and temporal inconsistency.

The central contribution is the **decoupling of perception and control**. By using POGS purely as an object-centric geometric memory and extracting clean point clouds via PointNet++ [6], we remove the burden of visual representation learning from the policy entirely. The ACT module receives a compact, viewpoint-invariant geometric latent z encoding only the task-relevant object geometry, and predicts temporally smoothed action chunks in joint space. This design avoids the scene-centric data bottlenecks of ManiGaussian [5] and GWM [2], the multi-camera hardware requirements of RVT [3], and the viewpoint sensitivity of standard 2D ACT [10].

The experimental implementation on the OpenManipulator-X validates the core claim: by engineering the perception system rather than multiplying cameras, POGS-ACT achieves robust grasping under partial occlusion and randomised object positions. The transition from a fragile 2D baseline plagued by shadow-based hallucinations to a stable 3D geometric policy confirms the practical value of object-centric 3D representations for real-world manipulation.

Ongoing work targets language-conditioned task generalization, virtual view augmentation, and multi-robot scaling. POGS-ACT establishes a principled, modular, and practically motivated foundation for scalable robot learning that is simultaneously more data-efficient, more computationally tractable, and more robust to real-world perceptual challenges than the current state of the art.

Bibliography

[1] Anonymous. "Grasping Partially Occluded Objects Using Autoencoder-Based Point Cloud Inpainting". In: *arXiv preprint arXiv:2503.12549* (2025).

[2] Anonymous. "GWM: Towards Scalable Gaussian World Models for Robotic Manipulation". In: *arXiv preprint arXiv:2508.17600* (2025).

[3] Ankit Goyal et al. "RVT: Robotic View Transformer for 3D Object Manipulation". In: *Conference on Robot Learning (CoRL)*. 2023.

[4] Valentin N. Hartmann et al. "Point Clouds Matter: Rethinking the Impact of Different Observation Spaces on Robot Learning". In: *arXiv preprint arXiv:2402.02500* (2024).

[5] Guanxing Lu et al. "ManiGaussian: Dynamic Gaussian Splatting for Multi-Task Robotic Manipulation". In: *arXiv preprint arXiv:2403.08321* (2024).

[6] Charles R. Qi et al. "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[7] Minghan Qin et al. "LangSplat: 3D Language Gaussian Splatting". In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.

[8] Justin Yu et al. "Persistent Object Gaussian Splat (POGS) for Tracking Human and Robot Manipulation of Irregularly Shaped Objects". In: *arXiv preprint arXiv:2503.05189* (2025).

[9] Yanjie Ze et al. "3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations". In: *arXiv preprint arXiv:2403.03954* (2024).

[10] Tony Z. Zhao et al. "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware". In: *Proceedings of Robotics: Science and Systems (RSS)*. 2023.

[11] Ziwen Zhuang, Yunxi Liu, and Lin Shao. "Play it by Ear: Learning Skills amidst Occlusion through Audio-Visual Imitation Learning". In: *arXiv preprint arXiv:2205.14850* (2022).